Control Units

M. Sonza Reorda

Politecnico di Torino
Dip. di Automatica e Informatica



1
1

Summary

- **Introduction**
- **Control Units**
- **Hardwired Control Units**
- **Microprogrammed Control Units**

2
2

Introduction

The CPU (*Central Processing Unit*) performs instruction fetch and execution.

The CPU executes each instructions in one or more simple steps (called *micro-instructions*).

3

3

Instruction Execution

The execution of each instruction can be divided in two phases:

- **Fetching from memory the machine code of the instruction pointed by the Program Counter register, and updating of the Program Counter (*Fetch Phase*)**
- **Execution of the instruction (*Execution Phase*).**

4

4

Simple (Elementary) Operations

Every operation that a CPU performs, can be divided in 4 elementary types:

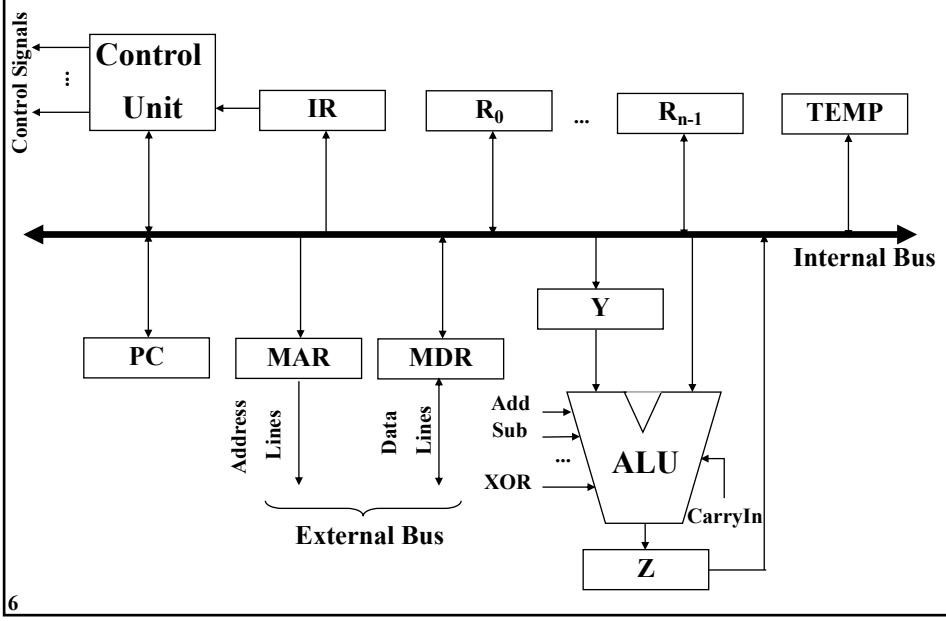
1. **Moving the value of a register to another register**
2. **Reading either a data or an instruction from memory and writing it in a register**
3. **Writing the value of a register into memory**
4. **Executing an arithmetic or logic operation and storing the result in a register.**

We will see in details how the above 4 types of operations can be implemented using a simple CPU model.

5

5

CPU Architecture



6

6

1 - Register to Register Transfer

We can assume that each register Ri holds an interface connected to the bus and controlled by two signals:

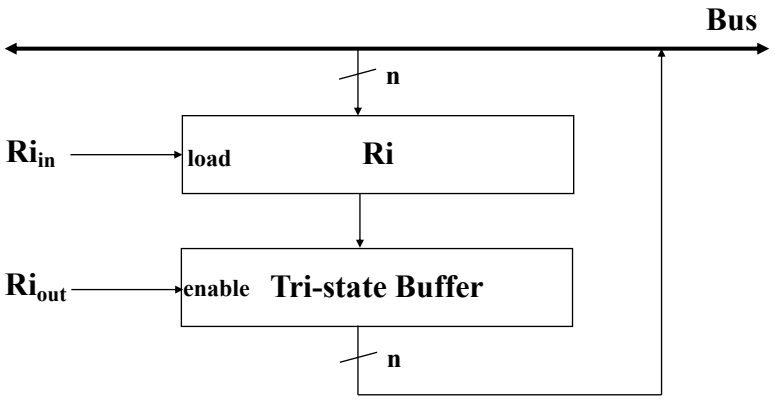
- **Ri_{in}, which causes the register to read the value from the bus**
- **Ri_{out}, which causes the contents of the register to be transferred to the bus; when the signal is not active, the register forces the value Z to the bus.**

For the correct operation of the bus it is essential that at any moment one and only one Ri_{out} signal is active.

7

7

Interface implementation



8

8

1 - Register to Register Transfer

It takes place acting appropriately on the interface signals of the registers connected to the internal bus of the CPU.

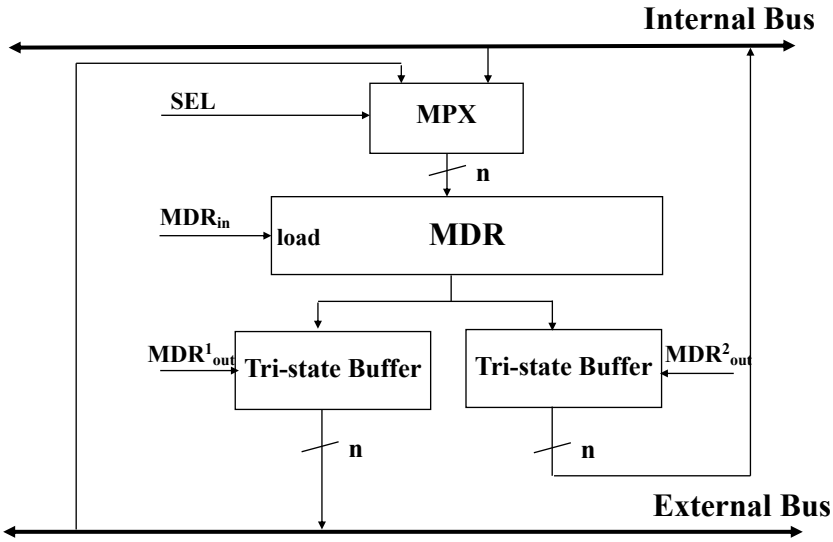
For example, to transfer the contents of register R1 to R4, you must:

- **activate $R1_{out}$: the value of R1 is placed on the bus**
- **activate $R4_{in}$: R4 reads the value on the bus and stores it.**

9

9

MDR connection



10

10

2 – Reading a word from memory

- **MAR ← R1**
- **Activate Read Signal**
- **Wait until MFC signal**
- **MDR ← External Bus**
- **R2 ← MDR**

The memory, after having received the read signal, places the requested data on the bus and enables the MFC signal (MFC = *Memory Function Completed*).

This procedure assumes an asynchronous bus.

11

11

3 – Writing a word into memory

- **MAR ← R1**
- **MDR ← R2, enable write signal**
- **Wait until MFC signal.**

12

12

4 - Logical or Arithmetic operation execution

In order to perform the sum of the registers R1 and R2 and put the result in R3, the CPU must activate the following control signals:

- **$R1_{out}$**
- **Y_{in}**
- **$R2_{out}$**
- **Add**
- **Z_{in}**
- **Z_{out}**
- **$R3_{in}$.**

13

13

Timings

Each signal must remain active for the time necessary to complete the operation controlled by it, taking into account the delays of the electronic components that need to be activate.

14

14

Example

Let consider the instruction

Add [R3], R1

which adds R1 to the content of the memory cell whose address is contained in R3, and places the result in R1.

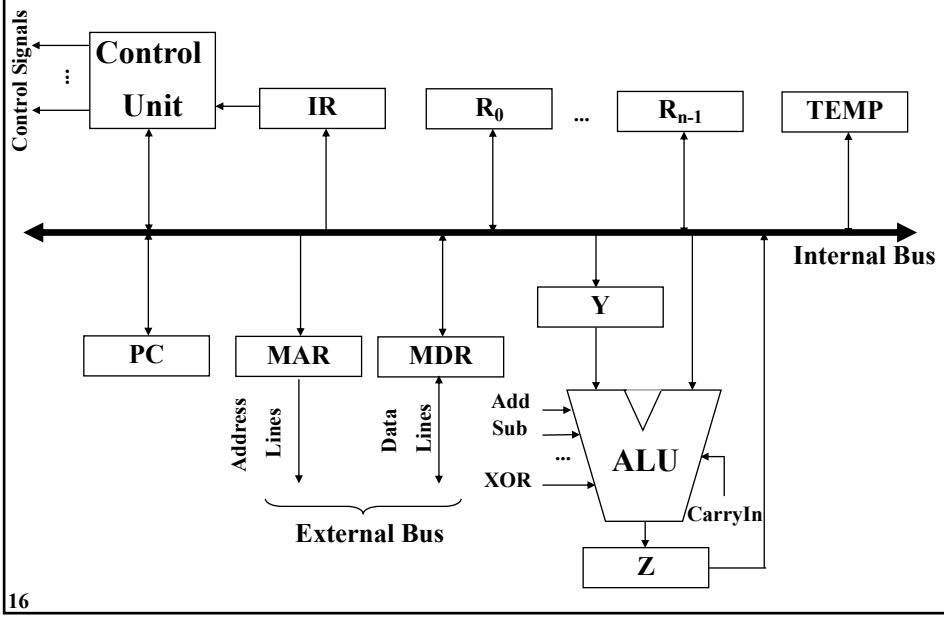
The control signals that must be activated are:

- PC_{out}, MAR_{in}, Read, Clear Y, Set Carry In to ALU, Add, Z_{in}
- Z_{out}, PC_{in}, wait until MFC
- MDR_{out}, IR_{in}
- $R3_{out}$, MAR_{in}, Read
- $R1_{out}$, Y_{in}, wait until MFC
- MDR_{out}, Add, Z_{in}
- Z_{out}, $R1_{in}$

15

15

CPU Architecture



16

16

Jump Instructions

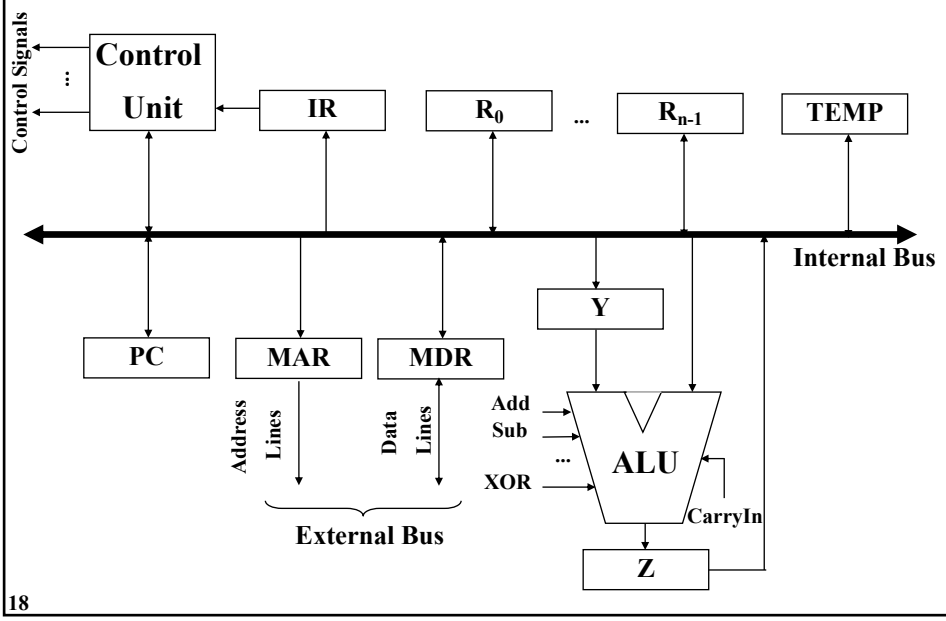
In order to execute unconditional jump instruction, the control signals that must be activated are:

- **PC_{out}, MAR_{in}, Read, Clear Y, Set Carry In to ALU, Add, Z_{in}**
- **Z_{out}, PC_{in}, wait until MFC**
- **MDR_{out}, IR_{in}**
- **PC_{out}, Y_{in}**
- **$(IR\ Offset)_{out}$, Add, Z_{in}**
- **Z_{out}, PC_{in}**

17

17

CPU Architecture



18

18

Conditional Branch

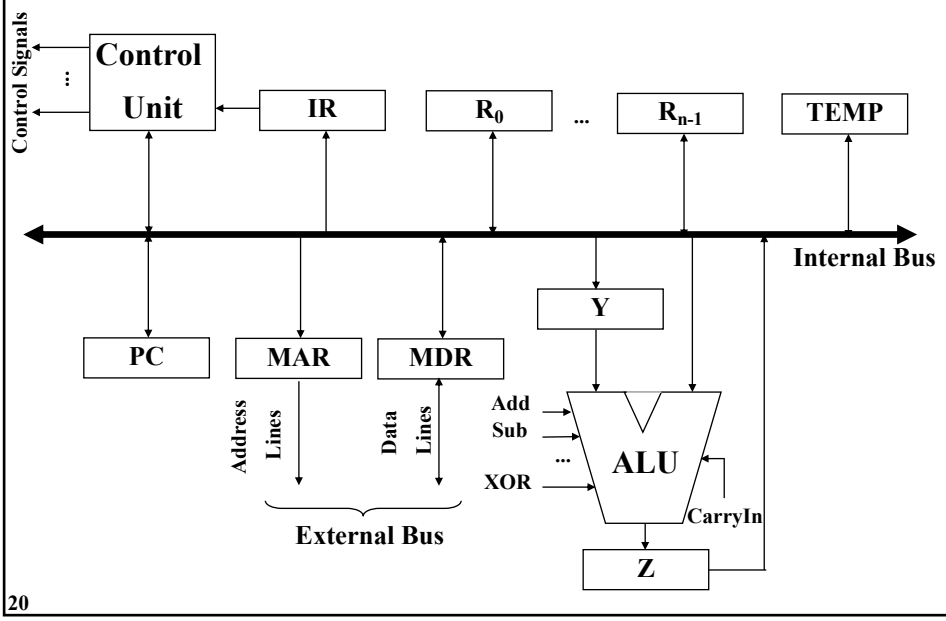
In order to execute a conditional branch instructions, the control signals that must be activated are:

- **PC_{out}, MAR_{in}, Read, Clear Y, Set Carry In to ALU, Add, Z_{in}**
- **Z_{out}, PC_{in}, wait until MFC**
- **MDR_{out}, IR_{in}**
- **PC_{out}, Y_{in}; if ZERO_FLAG=0 then End**
- **$(IR\ Offset)_{out}$, Add, Z_{in}**
- **Z_{out}, PC_{in}**

19

19

CPU Architecture



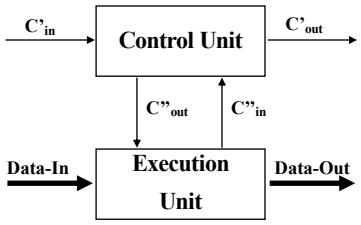
20

20

Control Units

Processors are composed of two parts:

- ***Execution Unit*** **(or *data-path*): it contains the registers, arithmetic-logic units, memories, etc.**
- ***Control Unit*** **(CU): it generates the control signals for the Execution Unit, based on the signals from the Execution Unit itself and from the outside**



21

21

How a Control Unit works

The Control Unit is a finite state machine which receives signals from outside (C'_{in}) and from the Execution Unit (C''_{in}).

Based on the values of these signals and on the current state, it generates control signals to the outside (C'out) and to the Elaboration Unit (C''out).

The complexity of the control unit is proportional to the product of the number of control signals (from outside and from Execution Unit) and the number of state variables.

22

22

Control Unit design

It usually starts with a description of its behavior, for example based on a state diagram.

Then, the corresponding hardware is designed.

There are two possible strategies:

- ***Hardwired control units*: the CU is treated as a normal finite state machine and the traditional methods for the design of sequential systems are applied**
- ***Microprogrammed control unit*: any operation that the controller has to perform corresponds to a microinstruction; microinstructions are stored in a memory, in a similar way to what happens with the programs.**

23

23

Goals

- **Minimize the hardware amount**
- **Maximize the execution speed**
- **Reduce design time.**

24

24

Hardwired CU design

The state of the control unit is given by the content of some flip flops.

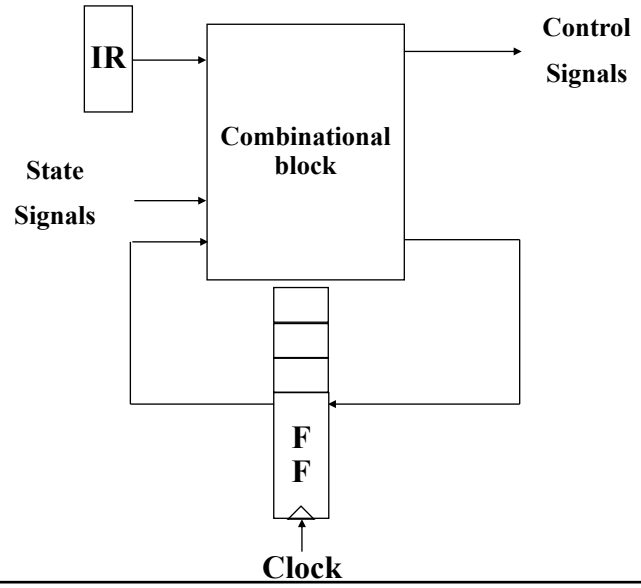
For each state and each possible combination of inputs (from outside and from the execution unit), the CU specifications provide the output values (control signals to the execution unit or to the outside) and the future state.

This can be modeled via a state table, which is then processed with the usual design methods for sequential circuits.

25

25

Architecture



26

26

Disadvantages

- The size of the state table may make it unusable
- The effort to design the CU may be unaffordable
- Once the table has been transformed to hardware, this does not have a regular structure, and the modifications are usually extremely complex.

27

27

Microprogrammed Control Units

- Introduction
- Features
- Vertical Microprogramming
- 8086 Control Units

28

28

Introduction

Microprogramming was proposed for the first time by M.V. Wilkes in 1950.

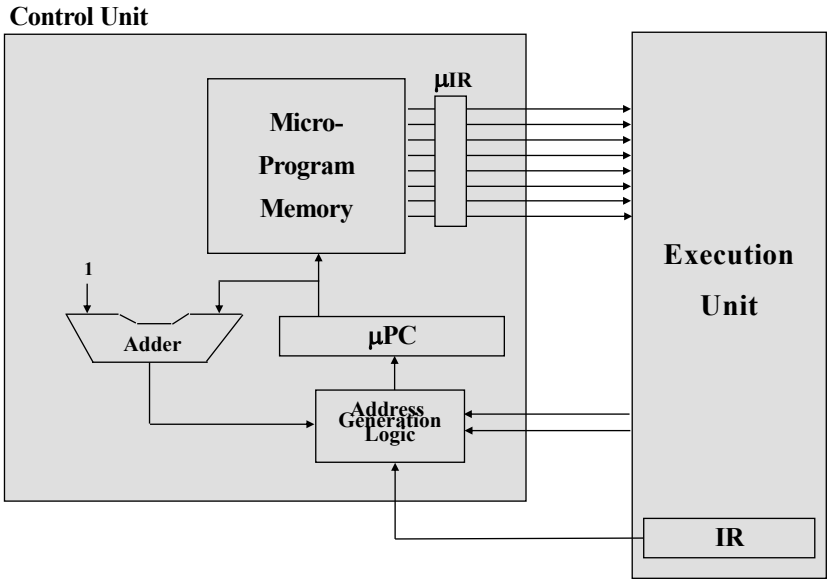
The set of control signals produced by the CU at a given time is called *Control Word*, or *microinstruction*.

Each instruction corresponds to a sequence of microinstructions, stored in a special memory (microprogram memory).

29

29

Architecture



30

30

Behavior

- The microprogram memory is accessed using the contents of the Micro Program Counter (μPC) as the address.
- The read word is loaded into the Micro Instruction Register (μIR).
- The content of the μIR drives the control signals to the execution unit and the Address Generation Logic.
- This logic generates a new address, based on the signals from μIR and from the outside (e.g., from the Instruction Register).

All these operations are performed in one clock cycle.

31

31

Next Microinstruction Address Generation

The address of the next microinstruction may be:

- the next one
- an address from the outside (e.g., the beginning of the microcode of a new instruction)
- a branch address (conditional or unconditional).

In the latter case the new address can be contained in the microinstruction itself.

32

32

Features of Microprogrammed Control Units

+ ***Flexibility*: modifying the behavior of an instruction or adding new ones simply involves changing the contents of the microprogram memory.**
- ***Speed*: the execution of a statement requires a number of microprogram memory accesses, and can therefore be slower than a hardwired CU.**
- ***Costs*: the presence of both the microprogram memory and its logic increases the cost in terms of hardware.**

33

33

Microinstruction format

It depends on how you solve two problems:

1. **determining the next microinstruction address**
2. **encoding the control signals.**

34

34

Address of the next microinstruction

If you do not have branch microinstructions, the address of the next micro instruction is given by just incrementing the previous one.

In the general case, the new address can be specified in several ways:

- each microinstruction has two additional fields that are used by conditional branch microinstructions to contain the addresses of two possible next microinstructions
- each microinstruction has an additional field, which is used by the conditional branch microinstructions to contain the possible jump address
- branch microinstructions have a different format then normal microinstructions.

35

35

Horizontal microinstructions

In the simplest case (*Horizontal* microinstructions), the microinstructions contain one bit for each control signal.

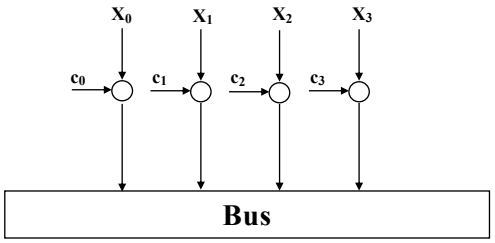
However:

- the length of the microinstructions may be excessive
- certain combinations of values of the control signals may never appear.

36

36

Example



Possible configurations:

0 0 0 1

0 0 1 0

0 1 0 0

1 0 0 0

37

37

Vertical microinstructions

They contain the information to be sent to the control signals in an *encoded* way.

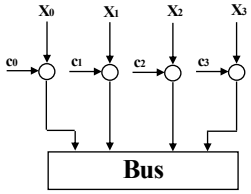
They have a smaller length than horizontal ones.

Intermediate solutions are also possible, where some groups of bits (*Control Field*) in a microinstruction, are encoded, only.

38

38

Example



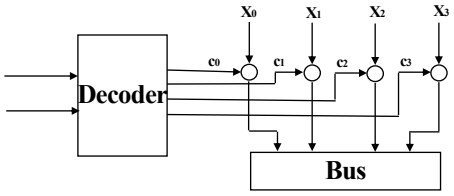
Possible configurations:

0 0 0 1

0 0 1 0

0 1 0 0

1 0 0 0



39

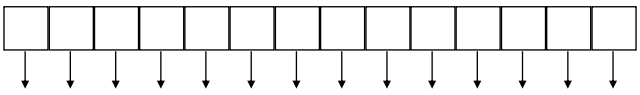
39

Alternative

Horizontal Microprogramming

Control fields

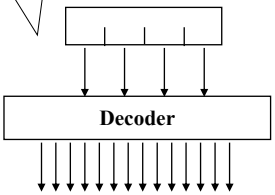
0 1 2 13



Control lines

Vertical Microprogramming

Single control field

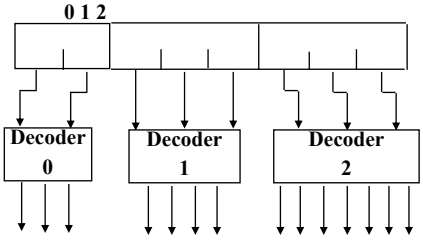


Control lines

Mixed Microprogramming

Control fields

0 1 2



Control lines

40

40

Compatibility

Two control signals are *compatible* if they are never activated in the same microinstruction.

They can therefore be associated with the same control field.

A *compatibility class* is a set of control signals which are compatible in pairs.

The control signals of a class of compatibility can be associated to the same set of control field, i.e., they can be encoded together.

41

41

Example

microinstruction	control signals
I_1	a b c g
I_2	a c e h
I_3	a d f
I_4	b c f

42

42

Example

microinstruction	control signals							
I_1	a	b	c				g	
I_2	a		c		e			h
I_3	a			d		f		
I_4		b	c	d		f		

Signals *a* and *f* are not compatible.

Signals *e*, *f* and *g* compose a compatibility class.

Signals *d* and *e* are compatible.

43

43

Examples of microprogrammed CPUs

- IBM System 360/370
- Intel 80x86
- Motorola 680x0

44

44

The 8088 control unit

General characteristics:

- **the control unit of 8088 (such as the one of 8086) is partly hardwired and partly microprogrammed**
- **microinstructions width is equal to 21 bits**
- **the format is vertical**
- **the microcode memory contains 504 microinstructions.**

45

45